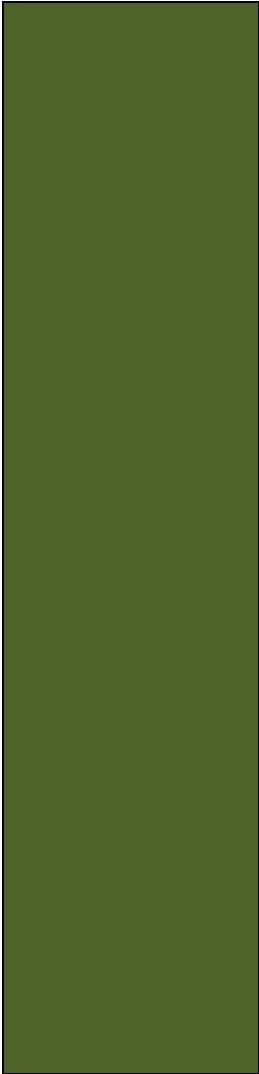


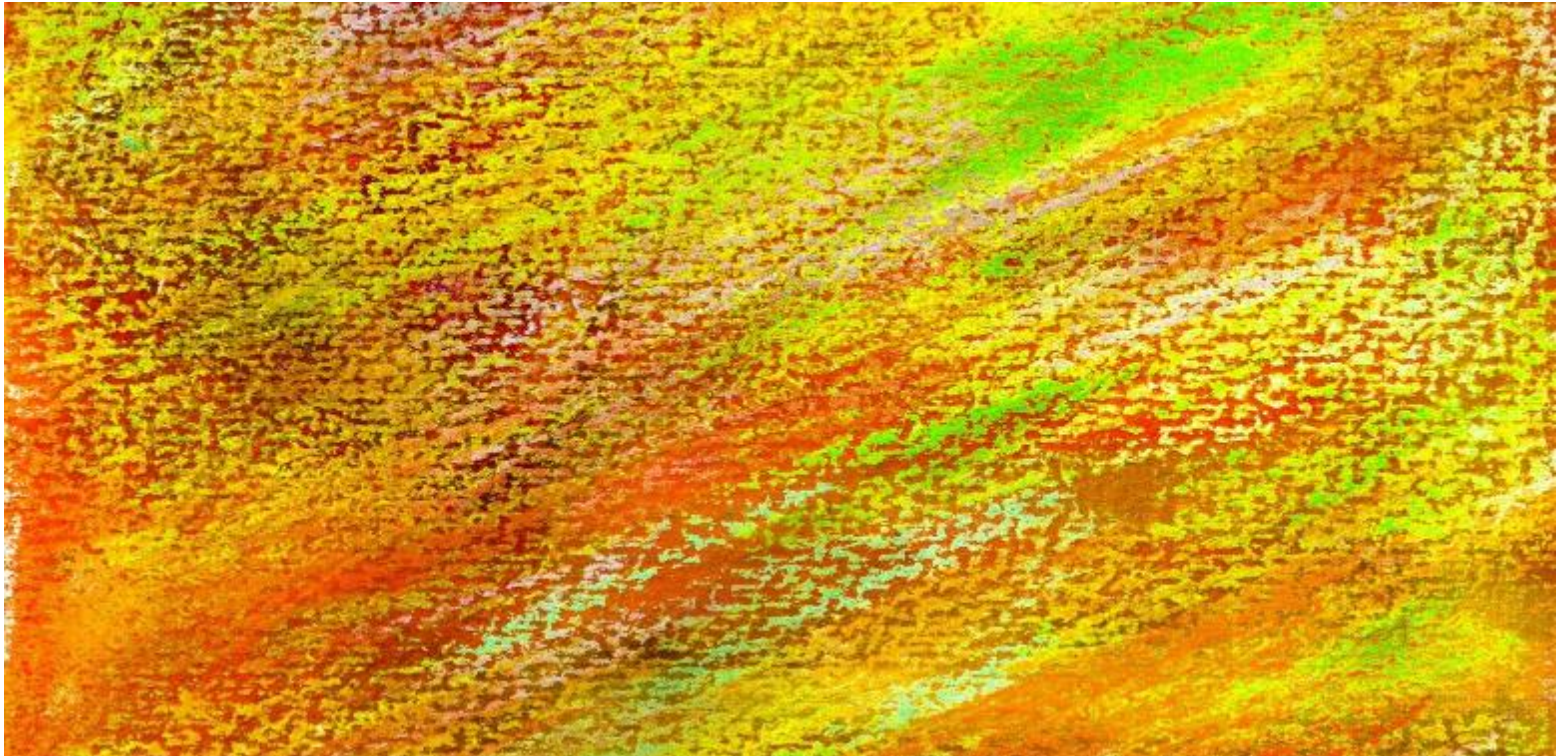


DAVID M. KROENKE and DAVID J. AUER
DATABASE CONCEPTS, 7th Edition



Chapter Seven

Database Issues





Chapter Objectives

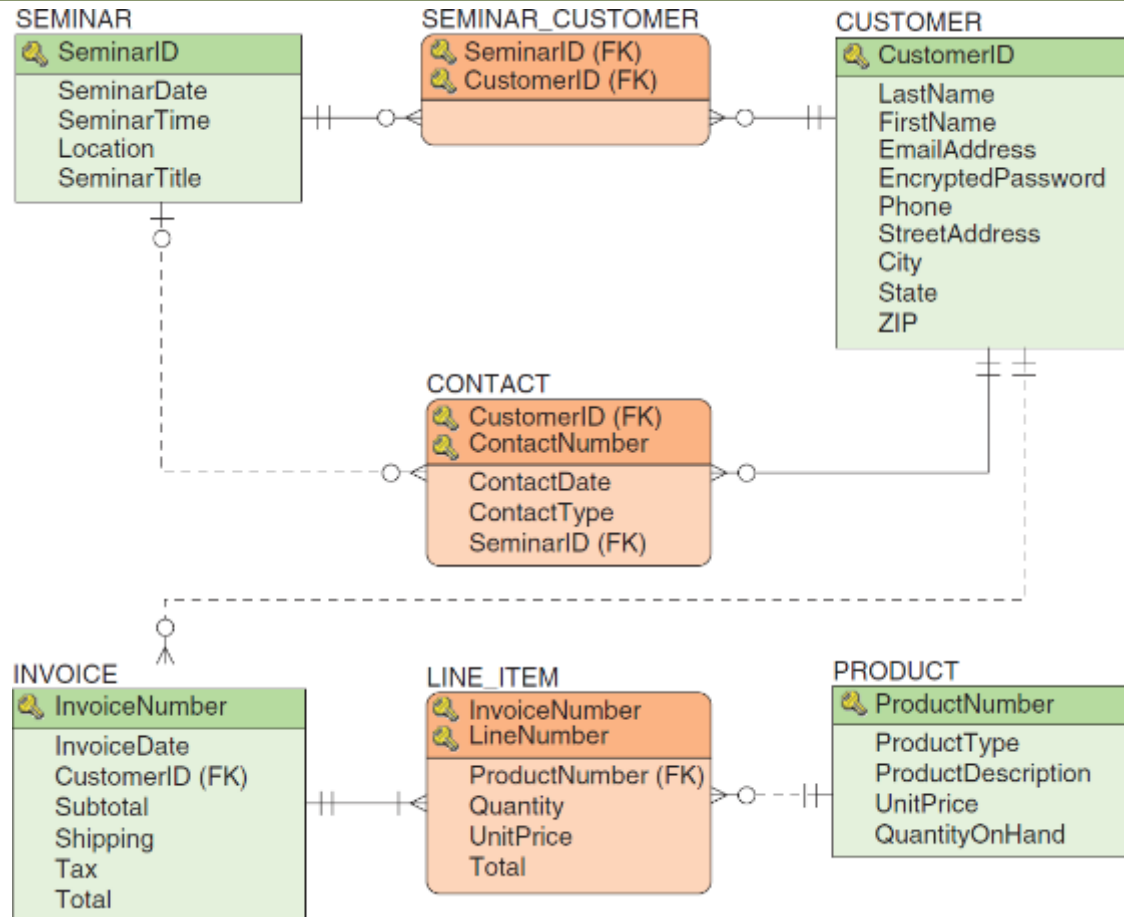
- Understand the need for and importance of database administration
- Learn different ways of processing a database
- Understand the need for concurrency control, security, and backup and recovery
- Learn about typical problems that can occur when multiple users process a database concurrently
- Understand the use of locking and the problem of deadlock
- Learn the difference between optimistic and pessimistic locking



Chapter Objectives (Cont'd)

- Know the meaning of *ACID transaction*
- Learn the four 1992 ANSI standard isolation levels
- Understand the need for security and specific tasks for improving database security
- Know the difference between recovery via reprocessing and recovery via rollback/rollforward
- Understand the nature of the tasks required for recovery using rollback/rollforward
- Know basic administrative and management DBA functions

Heather Sweeney Designs: Database Design





Heather Sweeney Designs: HSD Database in Microsoft SQL Server 2014

The HSD database object

The HSD table objects—*dbo* stands for *database owner*

The data in the **CUSTOMER** table

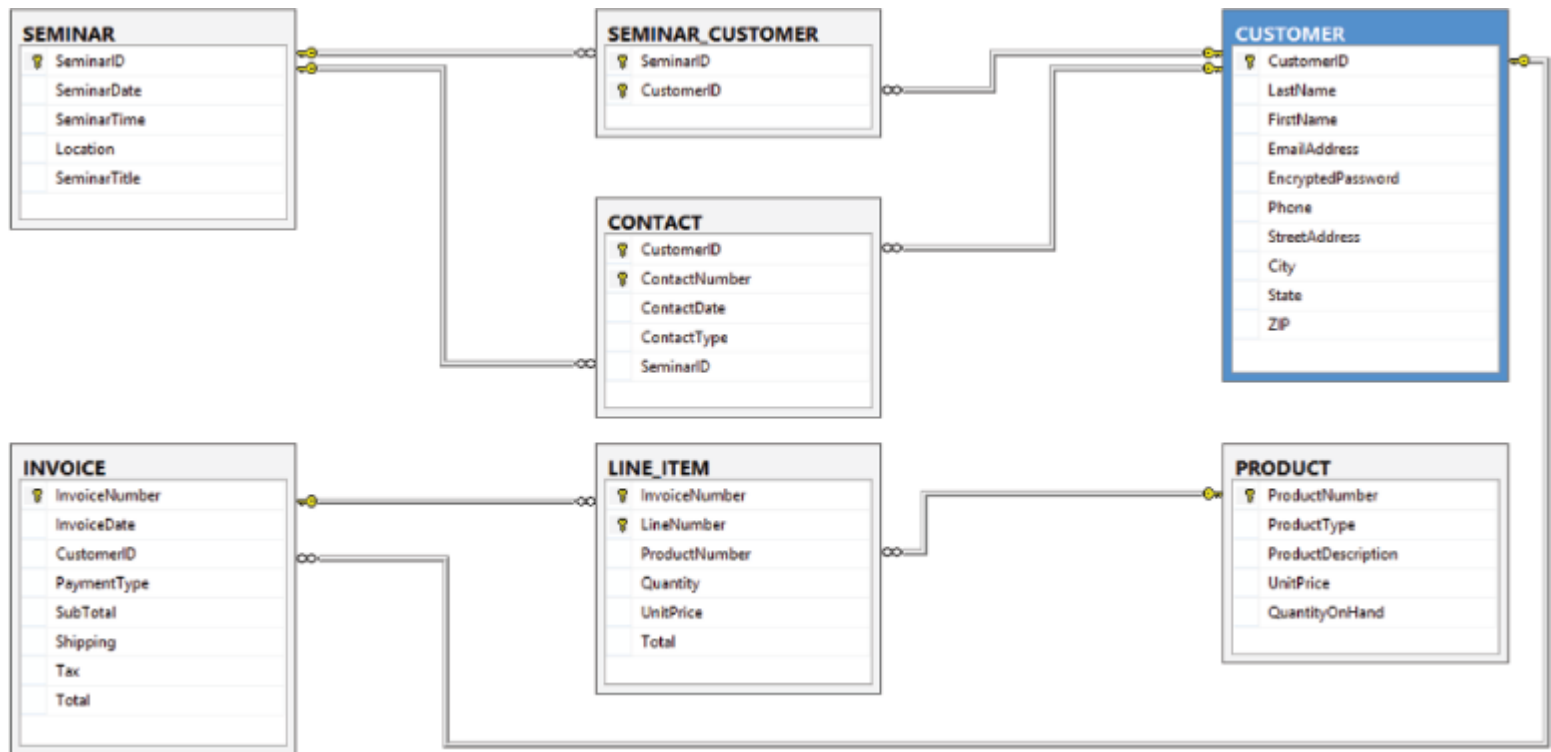
The screenshot displays the Microsoft SQL Server Management Studio interface. The Object Explorer on the left shows the database structure for 'WIN81-001\SQLEXPRESS (SQL Server 12.0)'. The 'HSD' database is expanded, showing 'Tables' and 'System Tables'. The 'CUSTOMER' table is selected. The main window shows the SQL query 'SELECT * FROM CUSTOMER;' and the results of the query, which is a table with 12 rows of customer data.

CustomerID	Last Name	First Name	Email Address	Encrypted Password	Phone	Street Address
1	Jacobs	Nancy	Nancy.Jacobs@somewhere.com	n46G9E	817-871-8123	1440 West Palm Drive
2	Jacobs	Charrel	Charrel.Jacobs@somewhere.com	b65TG0X	817-871-8234	1550 East Palm Drive
3	Able	Ralph	Ralph.Able@somewhere.com	m56GH0H	210-281-7987	123 Elm Street
4	Baker	Susan	Susan.Baker@elsewhere.com	PC93Bk-S	210-281-7876	456 Oak Street
5	Eagleton	Sam	Sam.Eagleton@elsewhere.com	bnvR44W8	210-281-7765	789 Pine Street
6	Foxtrot	Kathy	Kathy.Foxtrot@somewhere.com	aa@Y4GL	972-233-6234	11023 Elm Street
7	George	Sally	Sally.George@somewhere.com	LK9G2yF	972-233-6345	12034 San Jacinto
8	Hulett	Shawn	Shawn.Hulett@elsewhere.com	bu78VW3t	972-233-6456	13045 Flora
9	Pearson	Bobbi	Bobbi.Pearson@elsewhere.com	kg6T2Q0p	512-974-3344	43 West 23rd Street
10	Ranger	Tony	Tony.Ranger@somewhere.com	bv3F9Qc4	512-974-4455	56 East 18th Street
11	Tyler	Jenny	Jenny.Tyler@somewhere.com	Yu4be77Z	972-233-6567	14056 South Envy Str
12	Wayne	Joan	Joan.Wayne@elsewhere.com	JW4TX0g	817-871-8245	1660 South Aspen Dnr

Figure 6-1: The HSD Database in Microsoft SQL Server 2014



Heather Sweeney Designs: HSD Database Diagram in SQL Server 2012



The Database Processing Environment

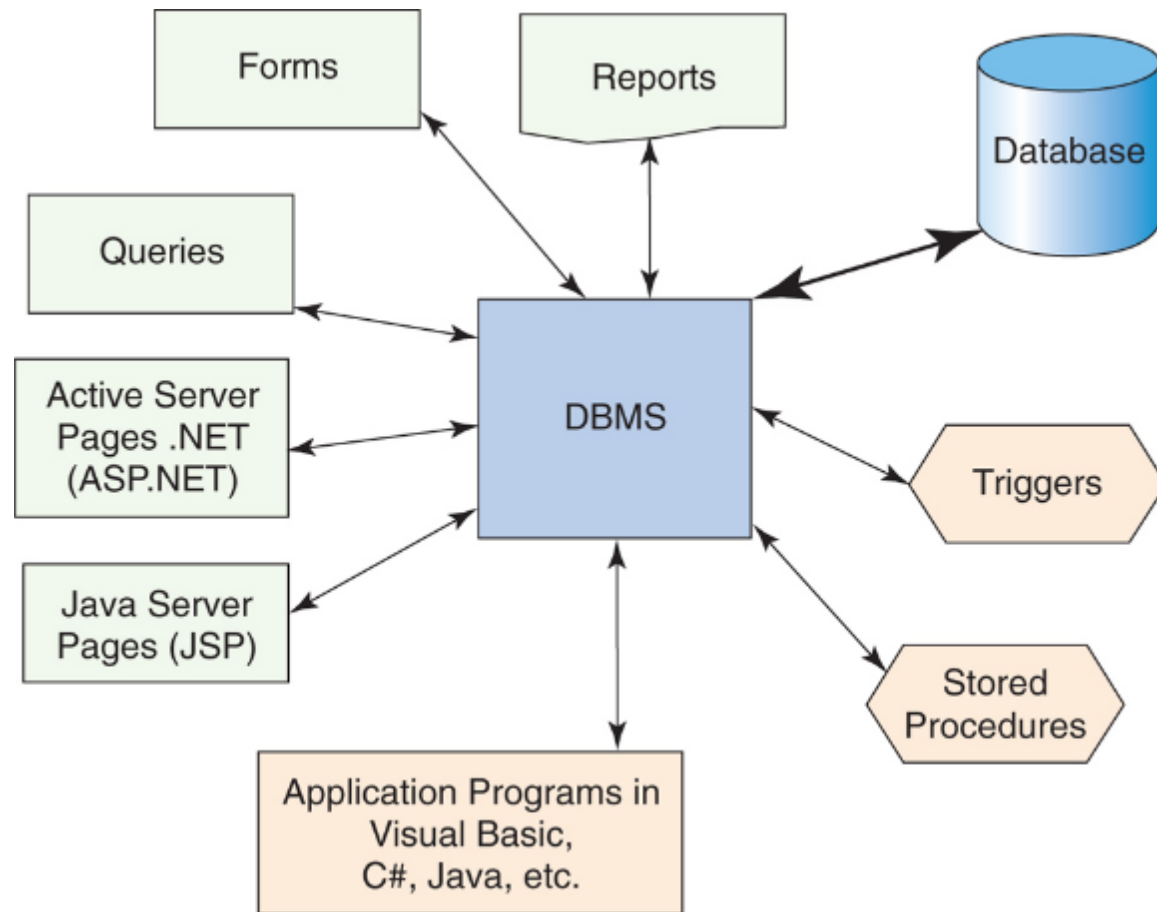


Figure 6-2: The Database Processing Environment



Control, Security and Reliability

- Three necessary database administration functions are:
 - Concurrency control
 - Security
 - Backup and Recovery



Concurrency Control

- **Concurrency control** ensures that one user's actions do not adversely impact another user's actions.
- At the core of concurrency is accessibility.
- In one extreme, data becomes inaccessible once a user touches the data.
 - This ensures that data that is being considered for update is not shown.
- In the other extreme, data is always readable.
 - The data is even readable when it is locked for update.



Concurrency Control (continued)

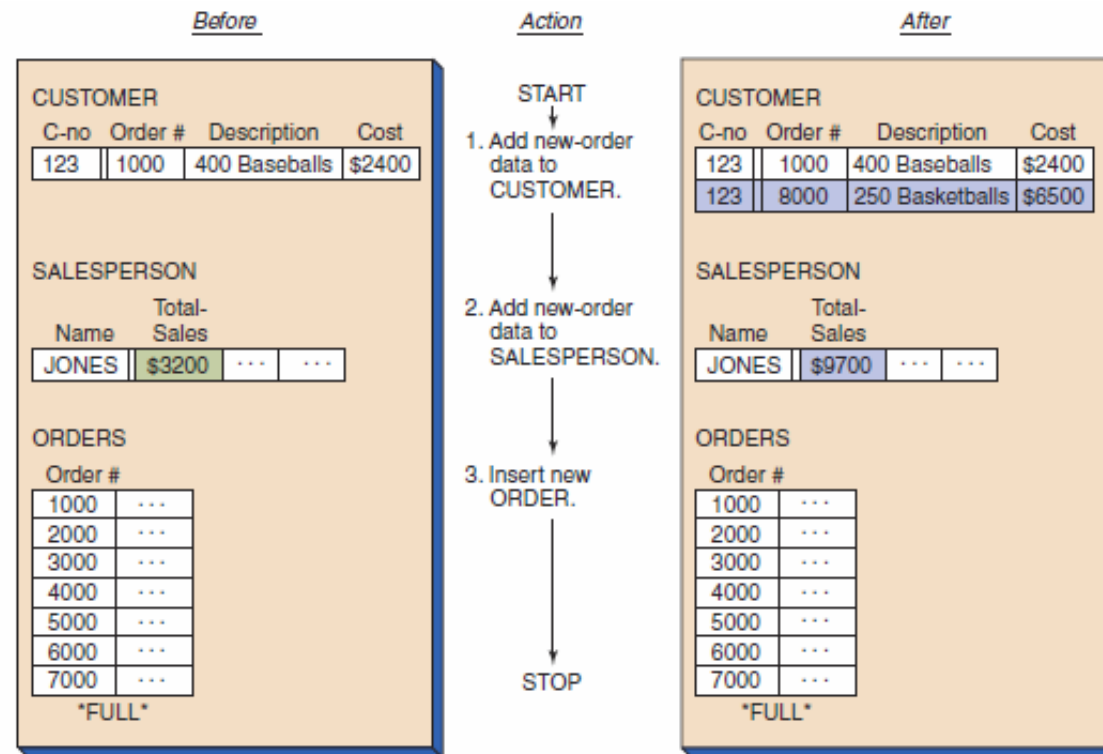
- Interdependency
 - Changes required by one user may impact others.
- Concurrency
 - People or applications may try to update the same information at the same time.
- Record retention
 - When information should be discarded



Need for Atomic Transactions

- A database operation typically involves several transactions.
- These transactions are *atomic* and are sometimes called **logical units of work (LUW)**.
- Before an operation is *committed* to the database, all LUWs must be successfully completed.
 - If one or more LUW is unsuccessful, a *rollback* is performed and no changes are saved to the database.

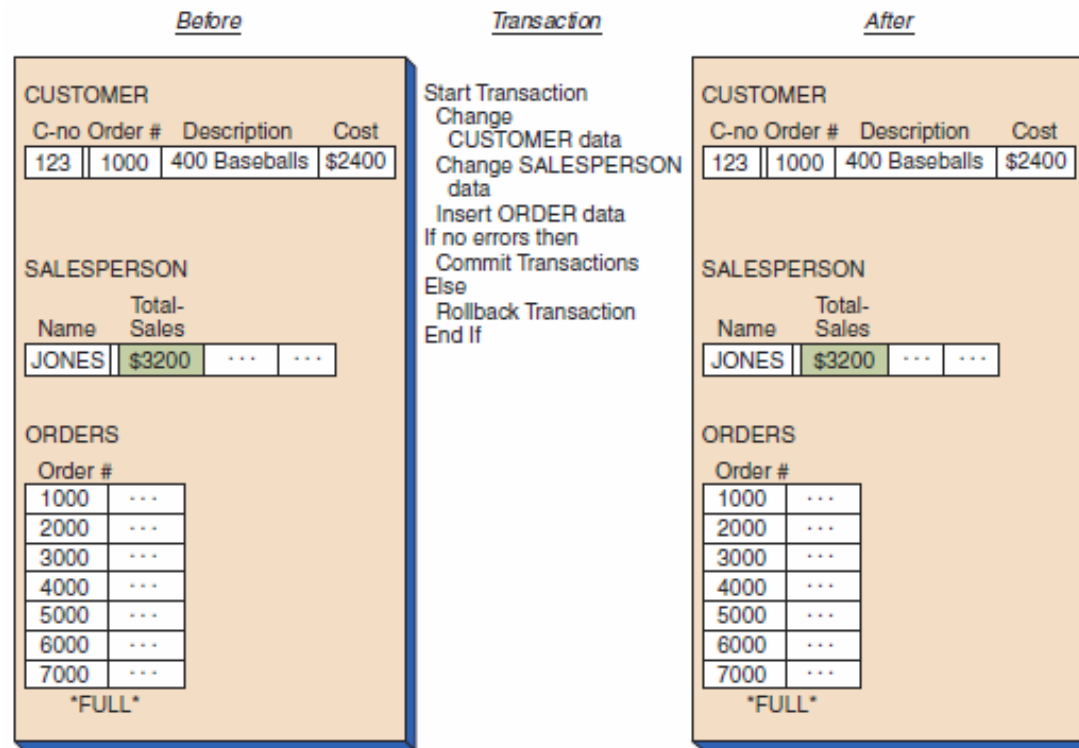
Transaction Example I



(a) Two of Three Activities Successfully Completed, Resulting in Database Anomalies

Figure 6-3: Comparison of the Results of Applying Serial Actions Versus a Multiple-Step Transaction

Transaction Example II



(b) No Change Made Because Entire Transaction Not Successful

Figure 6-3: Comparison of the Results of Applying Serial Actions Versus a Multiple-Step Transaction (Cont'd)



Concurrent Processing Example

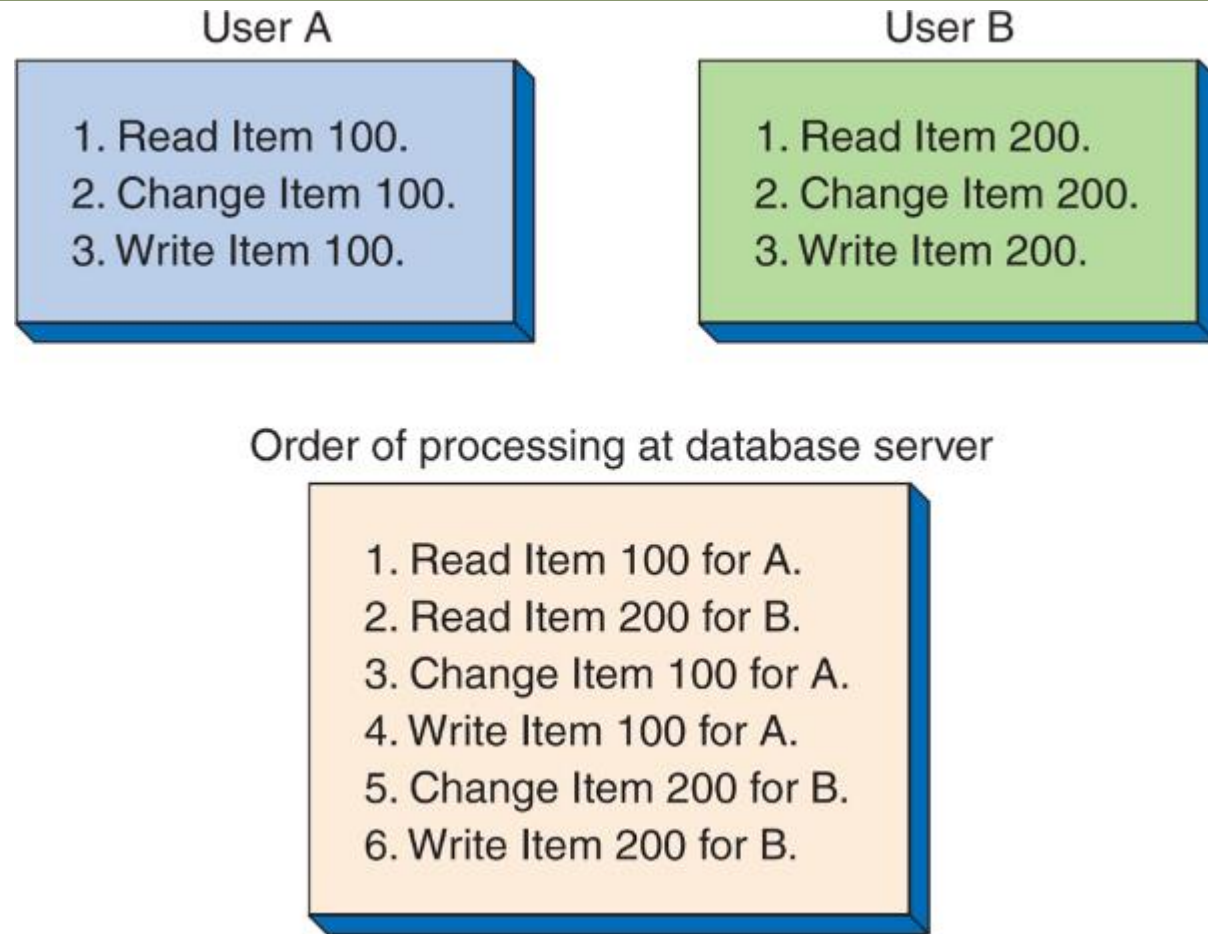


Figure 6-4: Example of Concurrent Processing of Two Users' Tasks

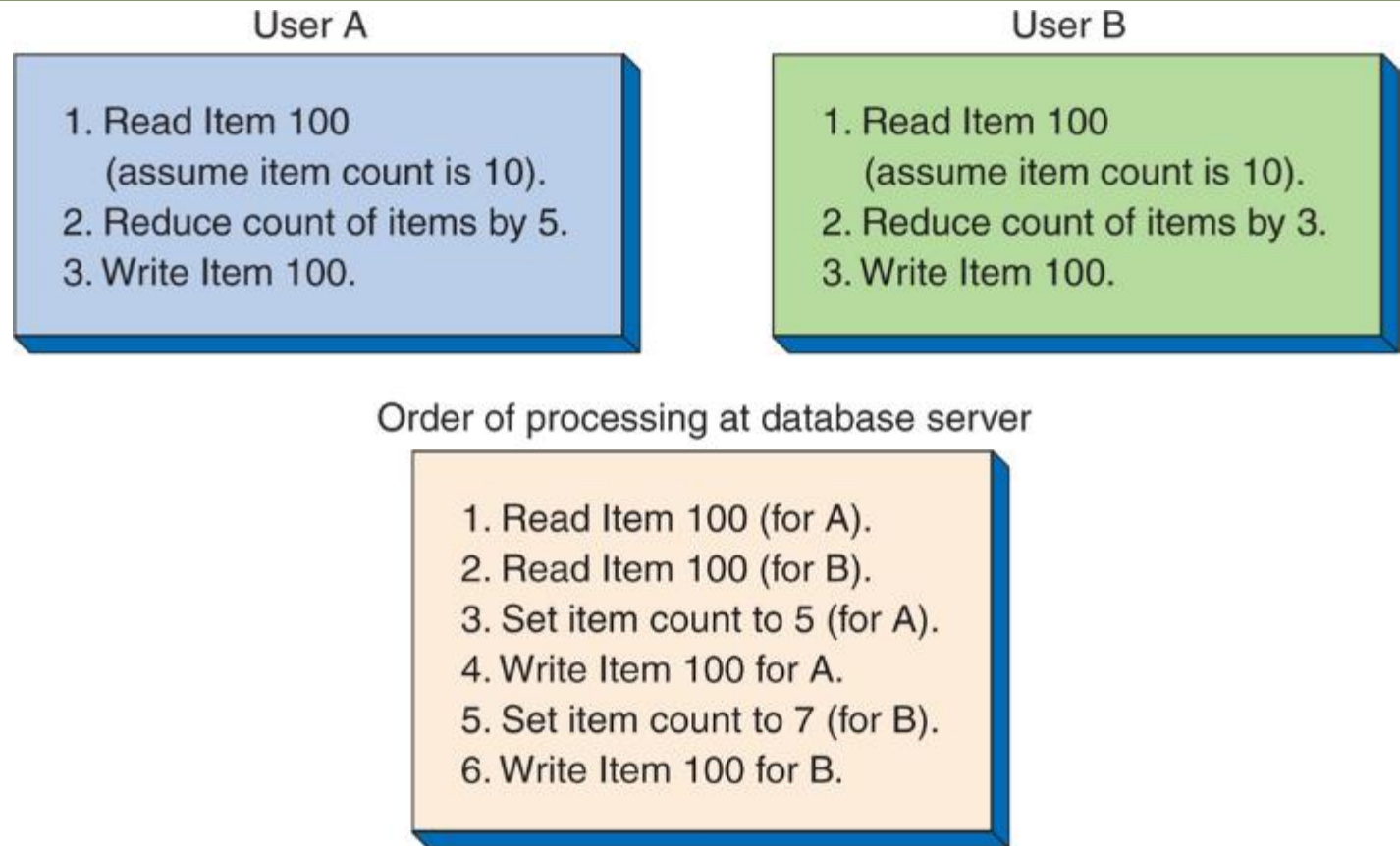


Lost Update Problem

- If two or more users are attempting to update the same piece of data at the same time, it is possible that one update may overwrite the other update



Lost Update Problem Example



Note: The change and write in steps 3 and 4 are lost.

Figure 6-5: Example of the Lost Update Problem



Concurrency Issues

- **Dirty reads**
 - The transaction reads a changed record that has not been committed to the database.
- **Inconsistent reads**
 - The transaction re-reads a data set and finds that the data has changed.
- **Phantom reads**
 - The transaction re-reads a data set and finds that a new record has been added.



Resource Locking

- To avoid concurrency issues, resource locking will disallow transactions from reading, modifying and/or writing to a data set that has been *locked*.



Implicit versus Explicit Resource Locking

- **Implicit locks** are issued automatically by the DBMS based on an activity.
- **Explicit locks** are issued by users requesting exclusive rights to the data.

Concurrent Processing with Explicit Locking Example

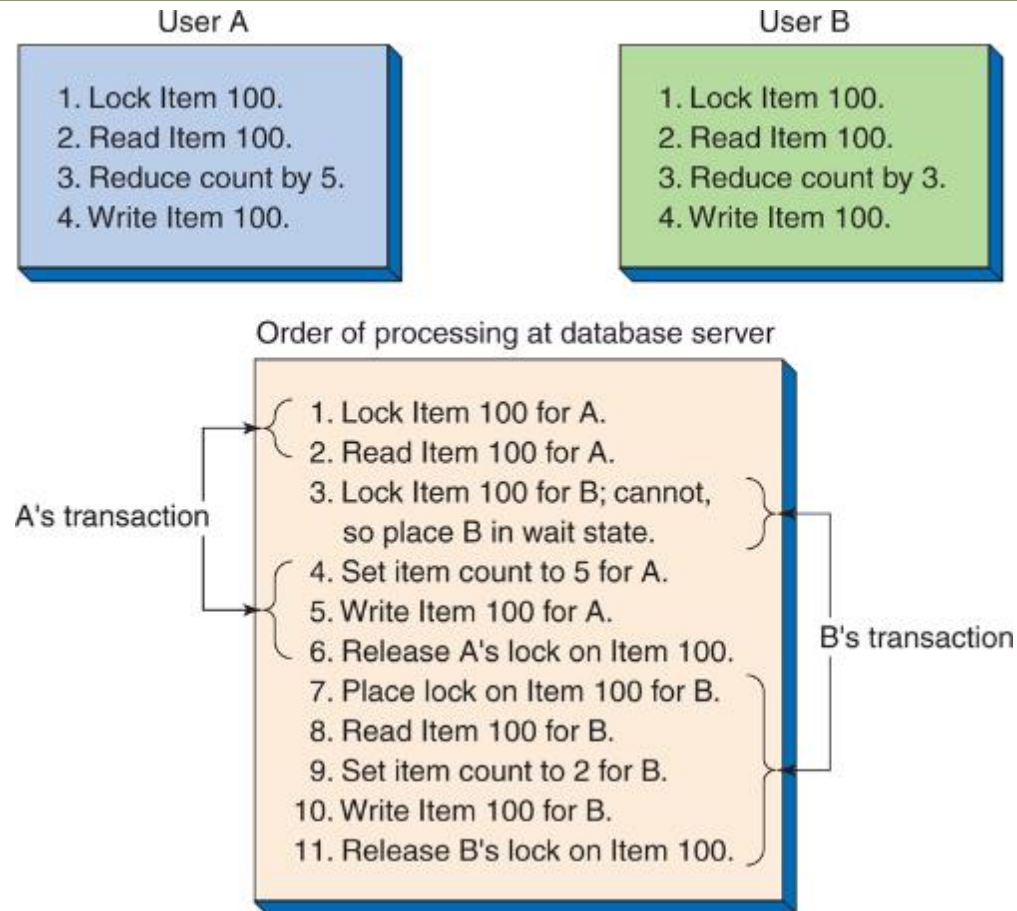


Figure 6-6: Example of Concurrent Processing with Explicit Locks



Serializable Transactions

- When two or more transactions are processed concurrently, the results in the database should be logically consistent with the results that would have been achieved had the transactions been processed in an *arbitrary serial fashion*.
- A scheme for processing concurrent transactions in this way is said to be **serializable**.



Two-Phased Locking

- One way to achieve serializable transactions is by using *two-phased locking*.
- **Two-phased locking** lets locks be obtained and released as they are needed.
 - A **growing phase**, when the transaction continues to request additional locks
 - A **shrinking phase**, when the transaction begins to release the locks



Deadlock

- As a transaction begins to lock resources, it may have to wait for a particular resource to be released by another transaction.
- On occasions, two transactions may indefinitely wait on each another to release resources—This condition is known as a **deadlock** or the **deadly embrace**.



Deadlock Example

User A

1. Lock paper.
2. Take paper.
3. Lock pencils.

User B

1. Lock pencils.
2. Take pencils.
3. Lock paper.

Order of processing at database server

1. Lock paper for User A.
2. Lock pencils for User B.
3. Process A's requests; write paper record.
4. Process B's requests; write pencil record.
5. Put A in wait state for pencils.
6. Put B in wait state for paper.

** Locked **

Figure 6-7: Example of Deadlock



Optimistic Locking versus Pessimistic Locking

- **Optimistic Locking**

- Read data
- Process transaction
- Issue update
- Look for conflict
- **IF** no conflict occurred
 THEN commit transaction
- **ELSE** rollback and repeat transaction

- **Pessimistic Locking**

- Lock required resources
- Read data
- Process transaction
- Issue commit
- Release locks



Optimistic Locking Example

```
SELECT  PRODUCT.Name, PRODUCT.Quantity
FROM    PRODUCT
WHERE   PRODUCT.Name = 'Pencil'
```

OldQuantity = PRODUCT.Quantity

Set NewQuantity = PRODUCT.Quantity – 5

{process transaction – take exception action if NewQuantity < 0, etc.

Assuming all is OK: }

LOCK PRODUCT {at some level of granularity}

```
UPDATE  PRODUCT
SET      PRODUCT.Quantity = NewQuantity
WHERE   PRODUCT.Name = 'Pencil'
        AND PRODUCT.Quantity = OldQuantity
```

UNLOCK PRODUCT

{check to see if update was successful;
if not, repeat transaction}

Figure 6-8: Example of Optimistic Locking



Pessimistic Locking Example

LOCK PRODUCT {at some level of granularity}

SELECT PRODUCT.Name, PRODUCT.Quantity
FROM PRODUCT
WHERE PRODUCT.Name = 'Pencil'

Set NewQuantity = PRODUCT.Quantity – 5

{process transaction – take exception action if NewQuantity < 0, etc.

Assuming all is OK: }

UPDATE PRODUCT
SET PRODUCT.Quantity = NewQuantity
WHERE PRODUCT.Name = 'Pencil'

UNLOCK PRODUCT

{no need to check if update was successful}

Figure 6-9: Example of Pessimistic Locking



SQL Transaction Control Language (TLC)

- The **SQL BEGIN TRANSACTION** statement
 - The **SQL COMMIT TRANSACTION** statement
 - The **SQL ROLLBACK TRANSACTION** statement
- NOTE: Exact SQL syntax varies between DBMS products.



Marking Transaction Boundaries Example

```
BEGIN TRANSACTION:

SELECT  PRODUCT.Name, PRODUCT.Quantity
FROM    PRODUCT
WHERE   PRODUCT.Name = 'Pencil'

Old Quantity = PRODUCT.Quantity
Set NewQuantity = PRODUCT.Quantity - 5

{process part of transaction – take exception action if NewQuantity < 0, etc.}

UPDATE  PRODUCT
SET     PRODUCT.Quantity = NewQuantity
WHERE   PRODUCT.Name = 'Pencil'

{continue processing transaction} . . .

IF transaction has completed normally      THEN
    COMMIT TRANSACTION
ELSE
    ROLLBACK TRANSACTION
END IF

Continue processing other actions not part of this transaction . . .
```

Figure 6-10: Example of Marking Transaction Boundaries



Consistent Transactions

- Consistent transactions are often referred to by the acronym *ACID*.
 - Atommic
 - Consistent
 - Isolated
 - Durable

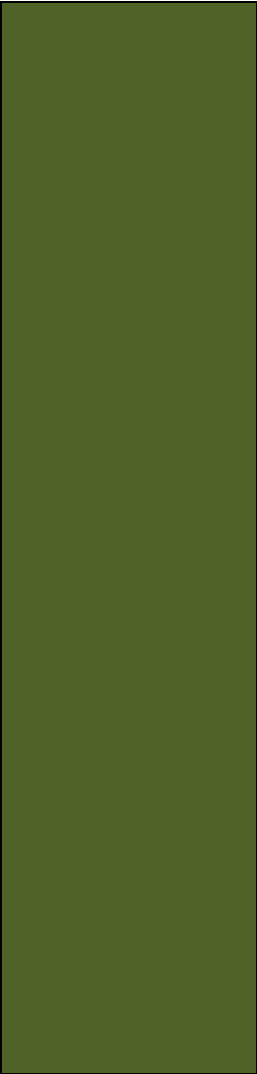


ACID: Atomic

- An **atomic** transaction is one in which all of the database actions occur or none of them do.
- A transaction consists of a series of steps. Each step must be successful for the transaction to be saved.
- This ensures that the transaction completes everything it intended to do before saving the changes.



ACID: Consistent

- 
- No other transactions are permitted on the records until the current transaction finishes.
 - This ensures that the transaction integrity has **statement level consistency** among all records.



ACID: Isolation

- Within multiuser environments, different transactions may be operating on the same data.
- As such, the sequencing of uncommitted updates, rollbacks, and commits continuously change the data content.
- The 1992 ANSI SQL standard defines four **isolation levels** that specify which of the concurrency control problems are allowed to occur.



ACID: Durable

- A **durable** transaction is one in which all committed changes are permanent.



Summary of Data Read Problems

Data Read Problem Type	Definition
Dirty Read	The transaction reads a row that has been changed, but the change has <i>not</i> been committed. If the change is rolled back, the transaction has incorrect data.
Nonrepeatable Read	The transaction rereads data that has been changed, and finds changes due to committed transactions.
Phantom Read	The transaction rereads data and finds new rows inserted by a committed transaction.

Figure 6-11: Summary of Data Read Problems

1992 ANSI SQL Isolation levels

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not possible	Not possible	Not possible
	Nonrepeatable Read	Possible	Possible	Not possible	Not possible
	Phantom Read	Possible	Possible	Possible	Not possible

Figure 6-12: Summary of Isolation Levels



Cursors

- A **cursor** is a pointer into a set of rows that are the result set from an SQL SELECT statement.
- Cursors are usually defined using SELECT statements.

```
DECLARE CURSOR TransCursor AS  
    SELECT *  
    FROM    SALE_TRANSACTION  
    WHERE   PurchasePrice > '10000';
```



Cursor Types

- **Forward only or scrollable**
- In SQL Server, for forward only or scrollable cursors, there are three types:
 - Static cursor
 - Keyset cursor
 - Dynamic cursor
- Other DBMS products may define a different set of cursors.



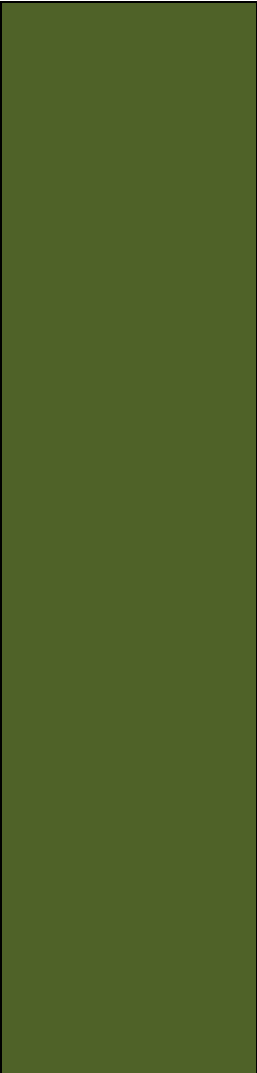
Summary of Cursor Types

Cursor Type	Description	Comments
Static	Application sees the data as they were at the time the cursor was opened.	Changes made by this cursor are visible. Changes from other sources are not visible. Backward and forward scrolling are allowed.
Keyset	When the cursor is opened, a primary key value is saved for each row in the recordset. When the application accesses a row, the key is used to fetch the current values for the row.	Updates from any source are visible. Inserts from sources outside this cursor are not visible (there is no key for them in the keyset). Inserts from this cursor appear at the bottom of the recordset. Deletions from any source are visible. Changes in row order are not visible. If the isolation level is dirty read, then committed updates and deletions are visible; otherwise, only committed updates and deletions are visible.
Dynamic	Changes of any type and from any source are visible.	All inserts, updates, deletions, and changes in recordset order are visible. If the isolation level is dirty read, then uncommitted changes are visible. Otherwise, only committed changes are visible.

Figure 6-13: Summary of Cursor Types



A Note on Cursor Types

- 
- Other DBMS products may define a different set of cursors.
 - In this case, the forward only cursor is considered a separate cursor type, and only a scrollable cursor may be static, keyset, or dynamic.

Database Security

- **Database Security** strives to ensure that
 - Only **authenticated** users
 - Perform **authorized** activities

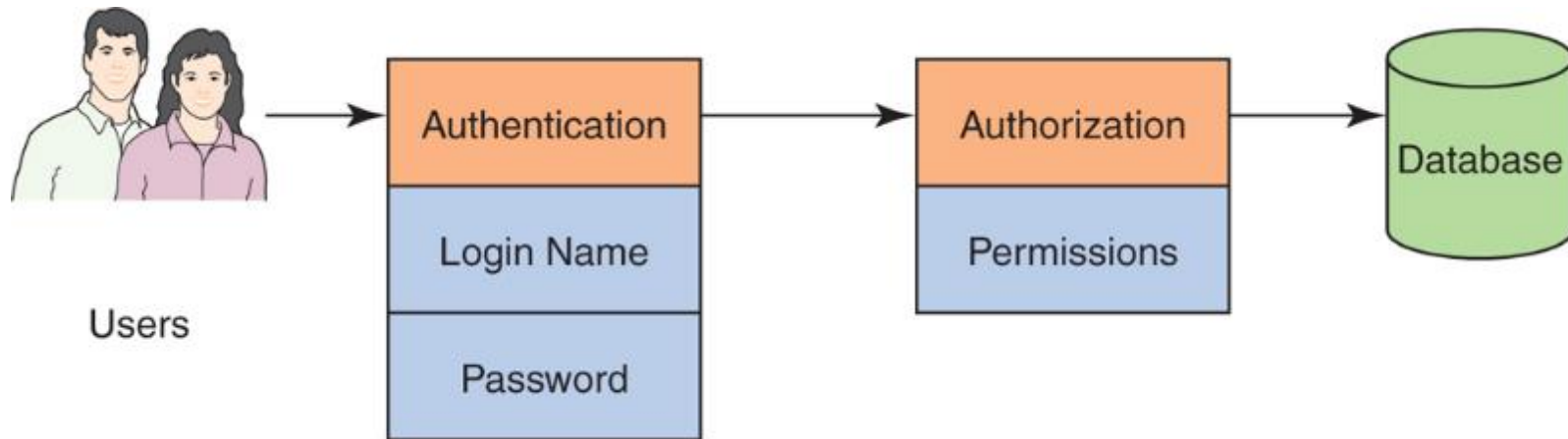
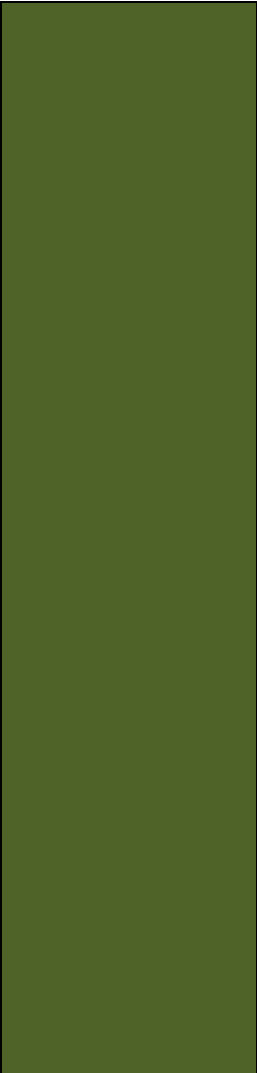


Figure 6-14: Database Security Authentication and Authorization



Processing Rights and Responsibilities

- 
- Processing rights define who is permitted to do what and when.
 - The individuals performing these activities have full responsibility for the implications of their actions.
 - Individuals are identified by a username and a password.

User Accounts in SQL Server 2014: Server Login Account

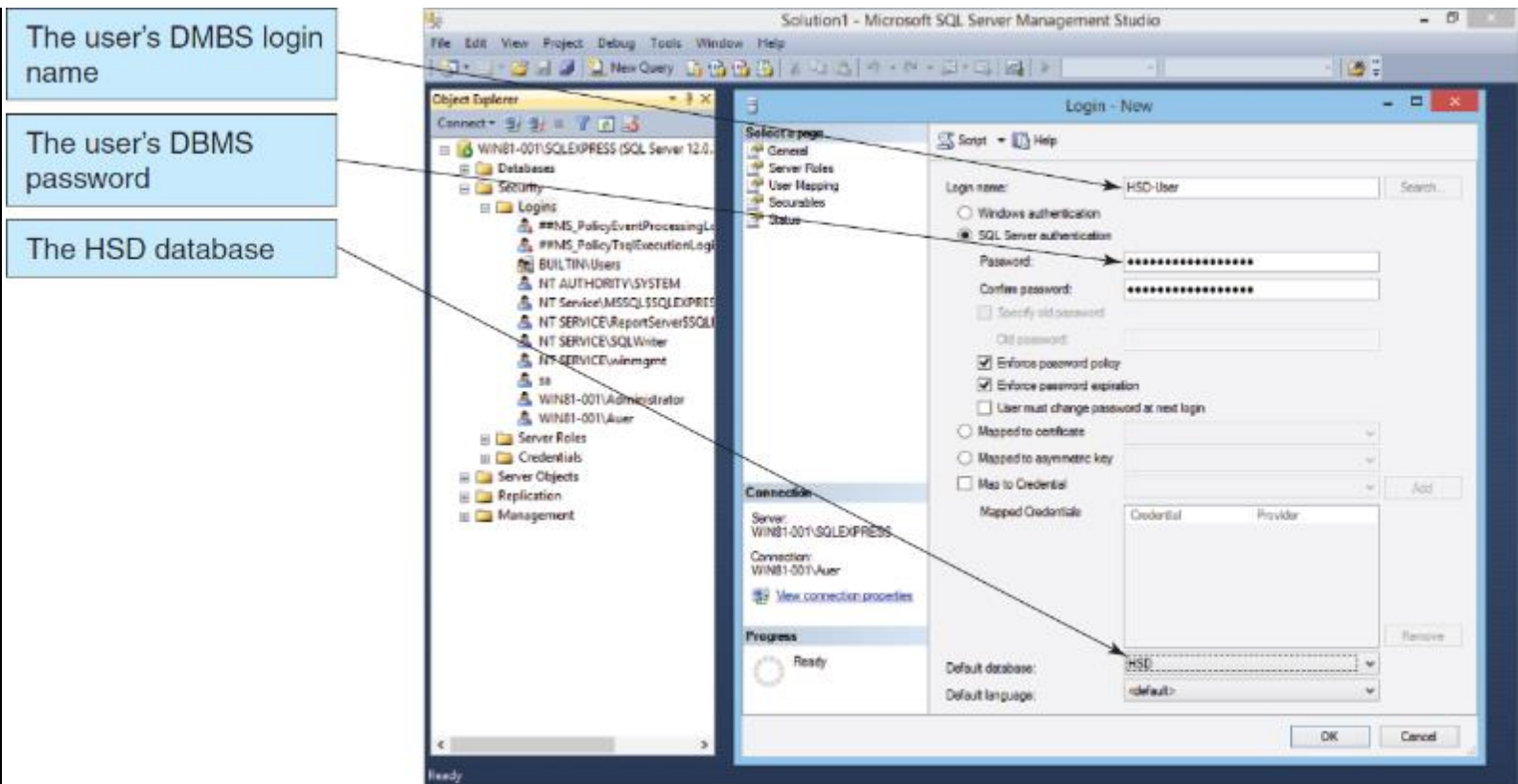


Figure 6-15: Creating the Database Server Login

A Model of DBMS Security

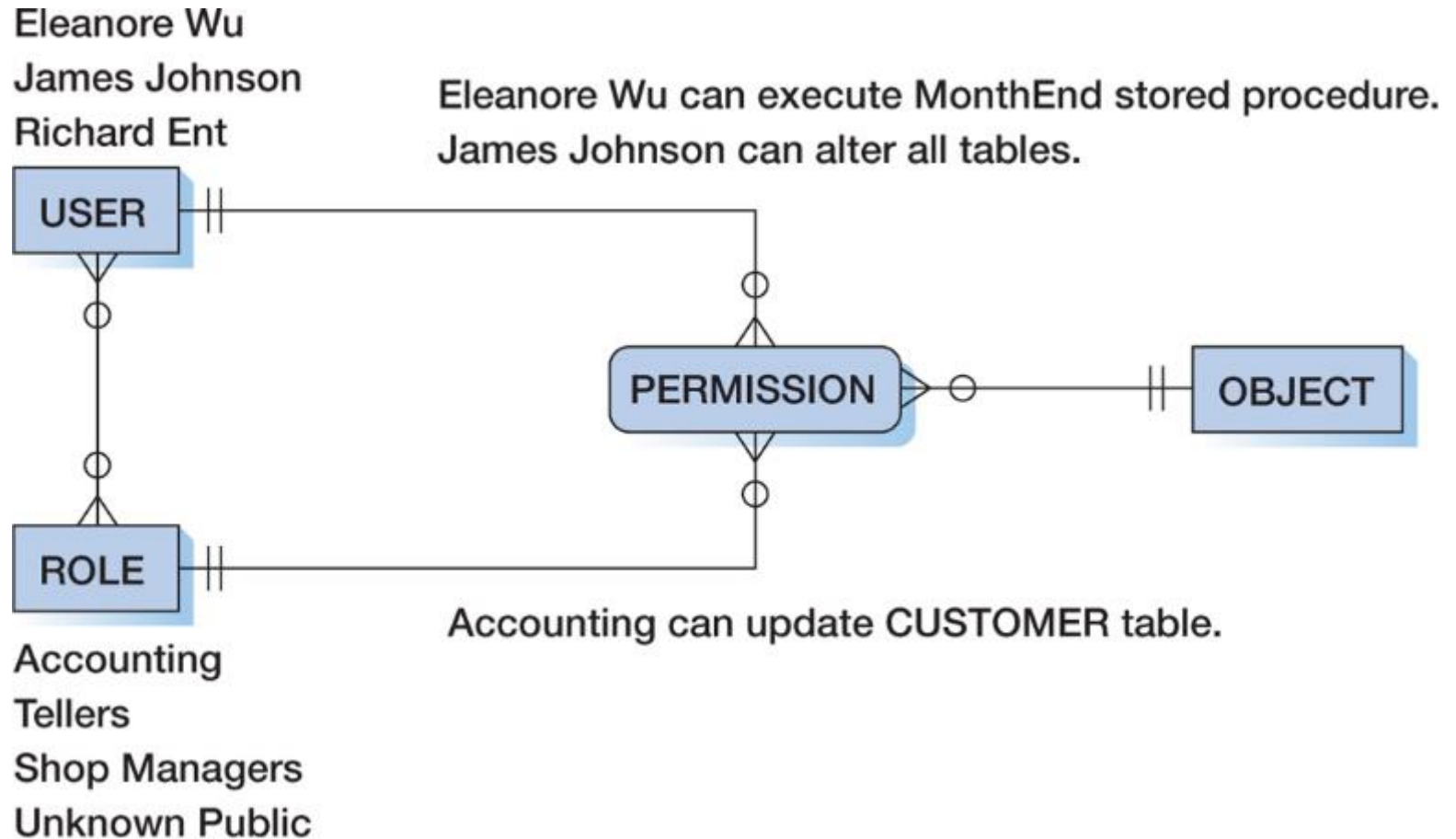


Figure 6-16: A Model of DBMS Security



Processing Rights at Heather Sweeney Designs

DATABASE RIGHTS GRANTED			
Table	Administrative Assistants	Management	System Administrator
SEMINAR	Read, Insert, Change	Read, Insert, Change, Delete	Grant Rights, Modify Structure
CUSTOMER	Read, Insert, Change	Read, Insert, Change, Delete	Grant Rights, Modify Structure
SEMINAR_CUSTOMER	Read, Insert, Change, Delete	Read, Insert, Change, Delete	Grant Rights, Modify Structure
CONTACT	Read, Insert, Change	Read, Insert, Change, Delete	Grant Rights, Modify Structure
INVOICE	Read, Insert, Change	Read, Insert, Change, Delete	Grant Rights, Modify Structure
LINE_ITEM	Read, Insert, Change, Delete	Read, Insert, Change, Delete	Grant Rights, Modify Structure
PRODUCT	Read, Insert, Change	Read, Insert, Change, Delete	Grant Rights, Modify Structure

Figure 6-17: Processing Rights at Heather Sweeney Designs

User Accounts in SQL Server 2014: Database User

The user name for the
HSD database

The user's DBMS login
name

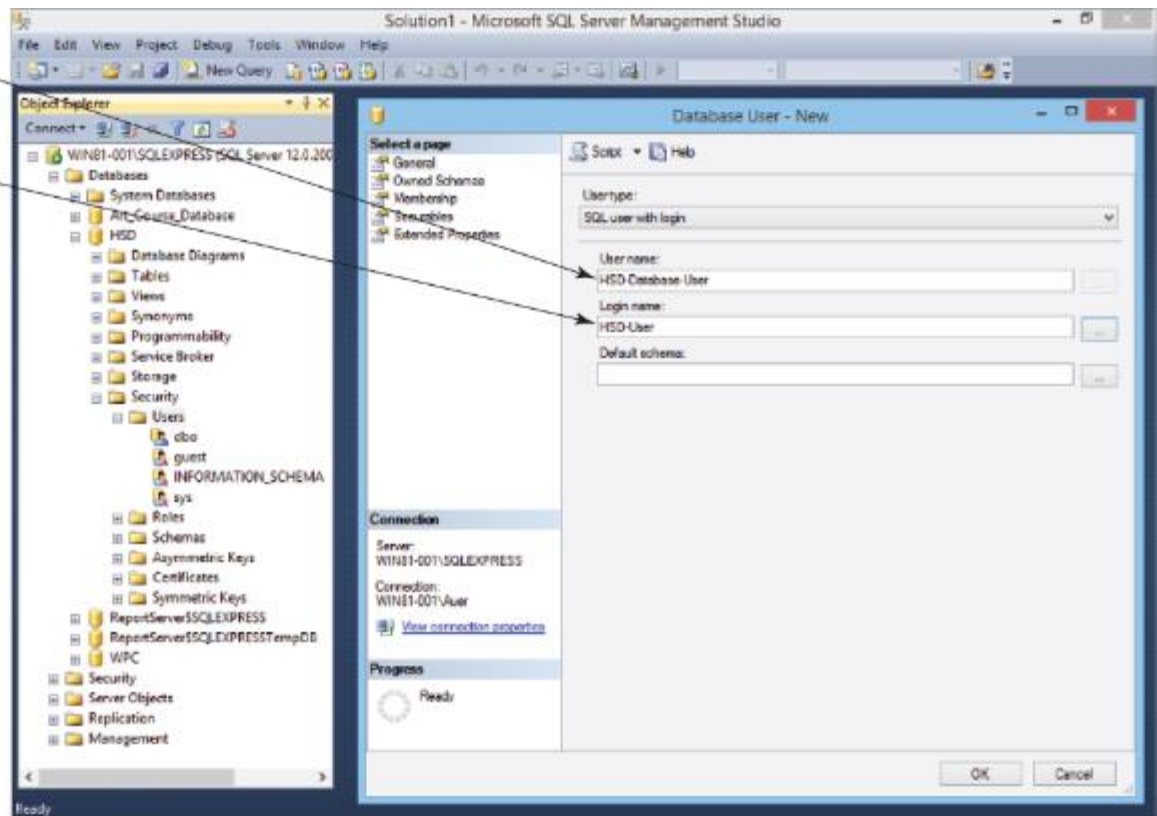
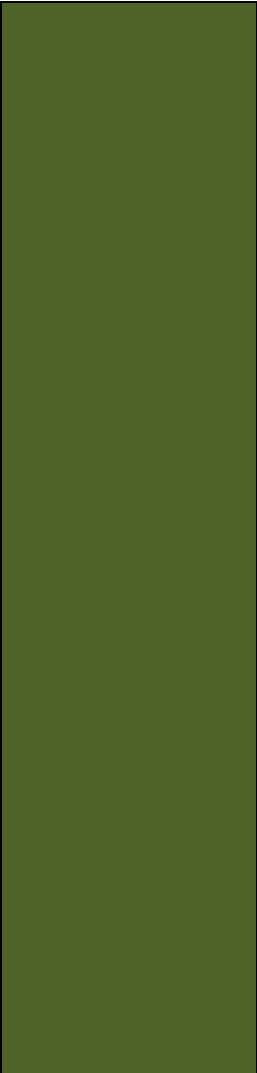


Figure 6-18: Creating the Database User Name



Granting Permissions

- 
- Database users are known as an individual and as a member of one or more roles.
 - Granting access and processing rights/privileges may be granted to an individual and/or a role.
 - Users possess the compilation of rights granted to the individual and all the roles for which they are members.

SQL Server 2014

Fixed Database Roles

Fixed Database Role	Database-Specific Permissions	DBMS Server Permissions
db_accessadmin	Permissions granted: ALTER ANY USER, CREATE SCHEMA Permissions granted with GRANT option: CONNECT	Permissions granted: VIEW ANY DATABASE
db_backupoperator	Permissions granted: BACKUP DATABASE, BACKUP LOG, CHECKPOINT	Permissions granted: VIEW ANY DATABASE
db_datareader	Permissions granted: SELECT	Permissions granted: VIEW ANY DATABASE
db_datawriter	Permissions granted: DELETE, INSERT, UPDATE	Permissions granted: VIEW ANY DATABASE
db_ddladmin	Permissions granted: <i>See SQL Server documentation</i>	Permissions granted: VIEW ANY DATABASE
db_denydatareader	Permissions <i>denied</i> : SELECT	Permissions granted: VIEW ANY DATABASE
db_denydatawriter	Permissions <i>denied</i> : DELETE, INSERT, UPDATE	Permissions granted: VIEW ANY DATABASE
db_owner	Permissions granted with GRANT option: CONTROL	Permissions granted: VIEW ANY DATABASE
db_securityadmin	Permissions granted: ALTER ANY APPLICATION ROLE, ALTER ANY ROLE, CREATE SCHEMA, VIEW DEFINITION	Permissions granted: VIEW ANY DATABASE

Note: For the definitions of each of the SQL Server permissions shown in the table, consult the SQL Server documentation.

Figure 6-19: SQL Server Fixed Database Roles

Assigning HSD-Database-User to the SQL Server 2014 db_datareader Role

The Database Role Properties – db_datareader dialog box

The HSD database user name HSD-Database-User

The Database Role db_datawriter

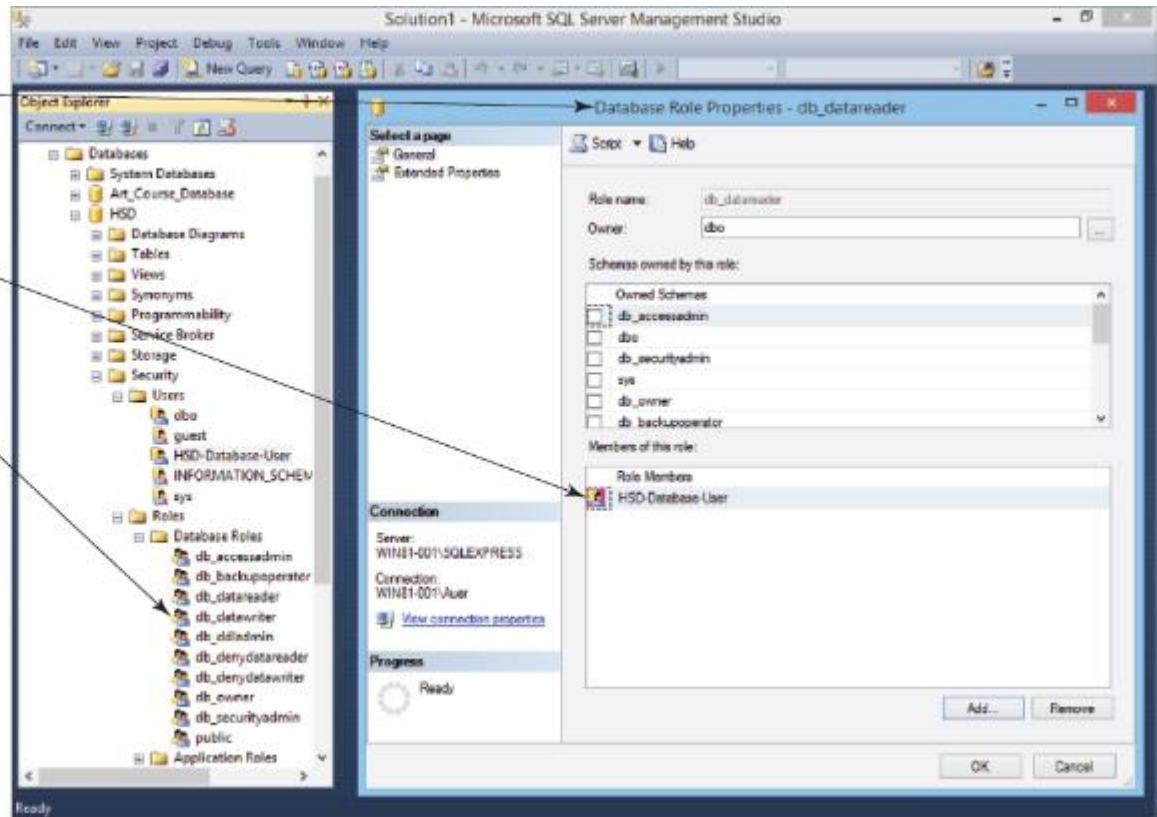


Figure 6-20: Assigning HSD-Database-User to the db_datareader Role

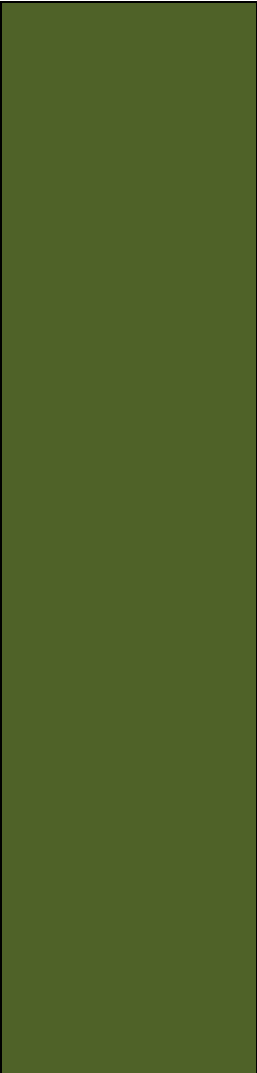


Database Security Guidelines

- Run the DBMS behind a firewall.
- Apply the latest operating system and DBMS service packs and patches.
- Limit DBMS functionality to needed features.
- Protect the computer that runs the DBMS.
- Manage accounts and passwords.



Database Backup and Recovery

- 
- Common causes of database failures
 - Hardware failures
 - Programming bugs
 - Human errors/mistakes
 - Malicious actions
 - As these issues are impossible to completely avoid, recovery procedures are essential.



Recovery via Reprocessing

- In **reprocessing**, all activities since the backup was performed are redone.
- This is a brute-force technique.
- This procedure is costly in the effort involved in re-entering the data.
- This procedure is risky in that human error is likely and in that paper record-keeping may not be accurate.



Recovery via Rollback and Rollforward

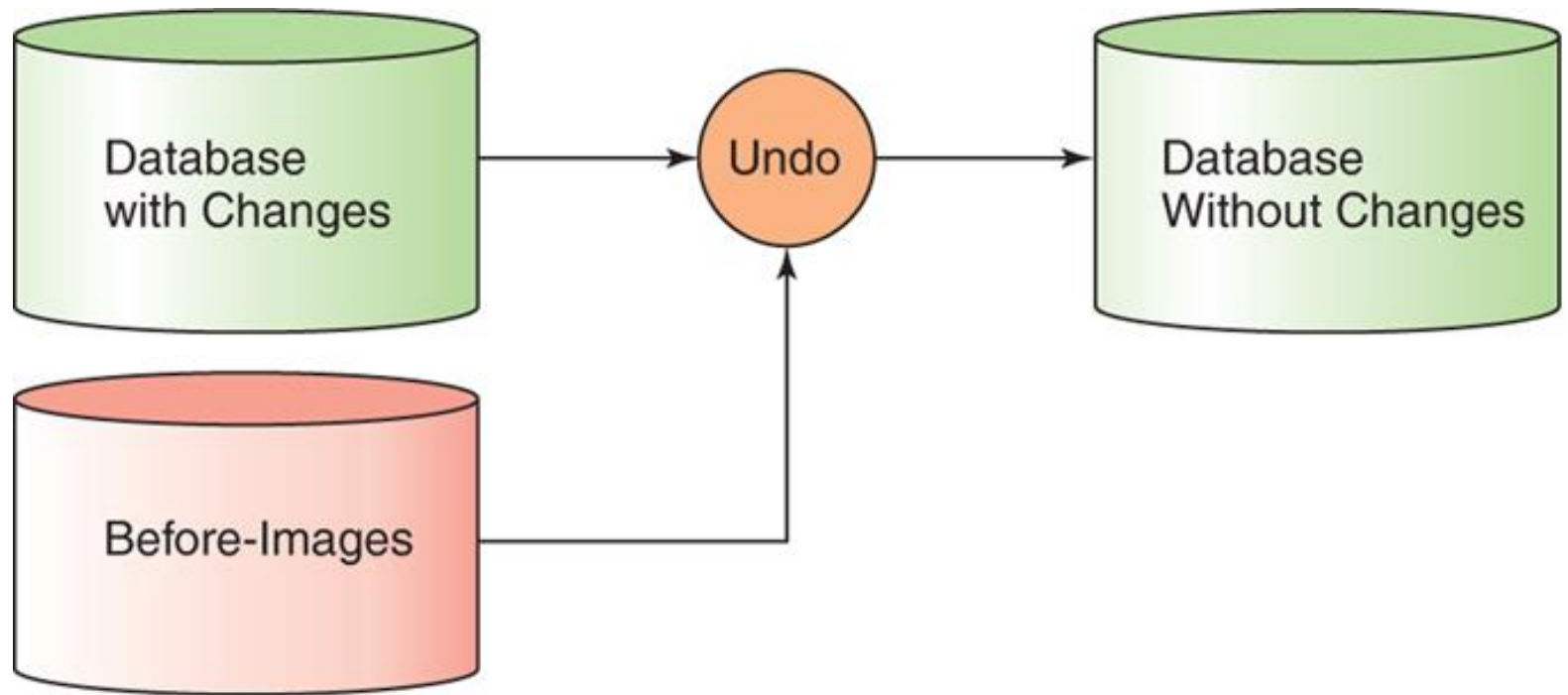
- Most database management systems provide a mechanism to record activities into a **log file**.
 - To undo a transaction the log must contain a copy of every database record before it was changed.
 - Such records are called **before-images**.
 - A transaction is undone by applying before-images of all its changes to the database.
 - To redo a transaction the log must contain a copy of every database record (or page) after it was changed.
 - These records are called **after-images**.
 - A transaction is redone by applying after-images of all its changes to the database.
- The log file is then used for recovery via *rollback* or *rollforward*.



Rollback

- **Rollback**
 - Log files save activities in sequence order.
 - It is possible to undo activities in reverse order that they were originally executed.
 - This is performed to correct/undo erroneous or malicious transaction(s) after a database is recovered from a full backup.

Rollback Example



(a) Rollback

Figure 6-22: Undo and Redo Transactions



Rollforward

- Rollforward
 - Activities recorded in the log files may be replayed.
 - In doing so, all activities are re-applied to the database.
 - This procedure is used to resynchronize restored database data by adding transactions to the last full backup.

Rollforward Example

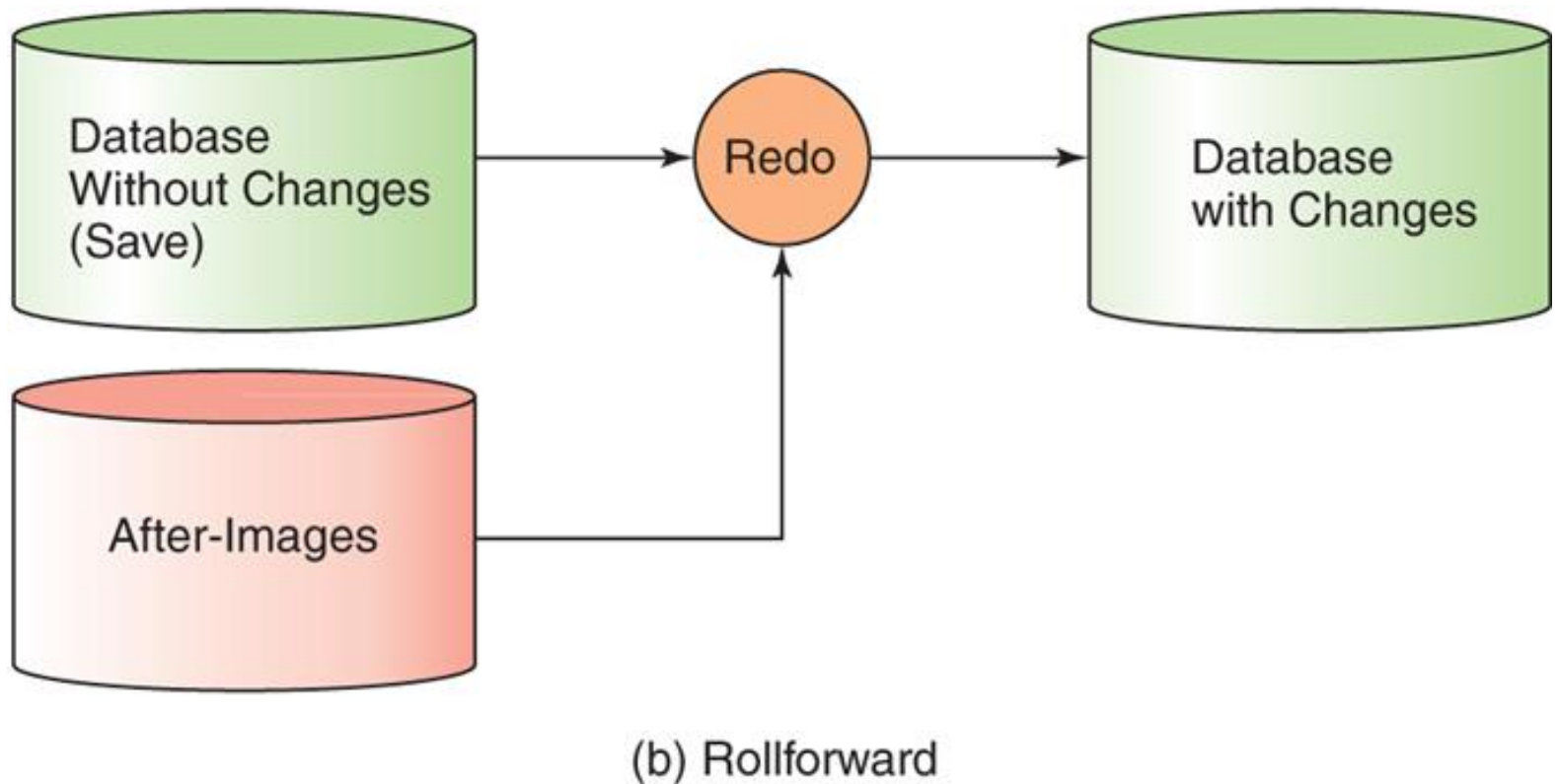


Figure 6-22: Undo and Redo Transactions (Cont'd)

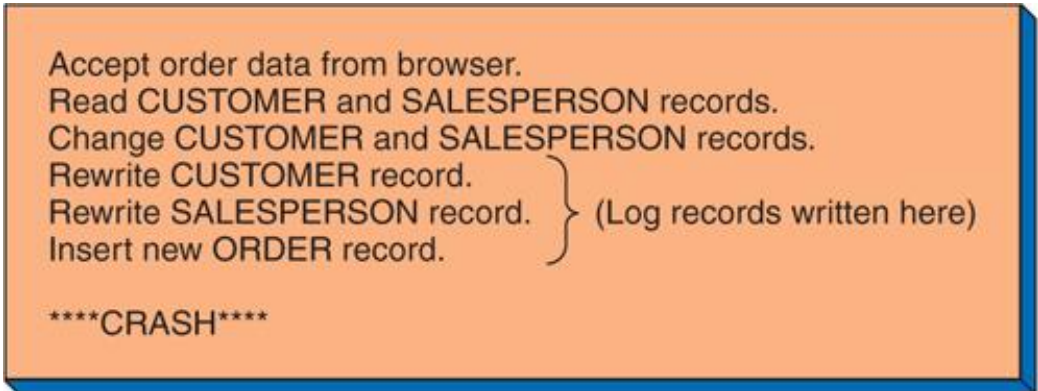
Example Transaction Log

Relative Record Number	Transaction ID	Reverse Pointer	Forward Pointer	Time	Type of Operation	Object	Before-Image	After-Image
1	OT1	0	2	11:42	START			
2	OT1	1	4	11:43	MODIFY	CUST 100	(old value)	(new value)
3	OT2	0	8	11:46	START			
4	OT1	2	5	11:47	MODIFY	SP AA	(old value)	(new value)
5	OT1	4	7	11:47	INSERT	ORDER 11		(value)
6	CT1	0	9	11:48	START			
7	OT1	5	0	11:49	COMMIT			
8	OT2	3	0	11:50	COMMIT			
9	CT1	6	10	11:51	MODIFY	SP BB	(old value)	(new value)
10	CT1	9	0	11:51	COMMIT			

Figure 6-23: Transaction Log Example



Recovery Example I



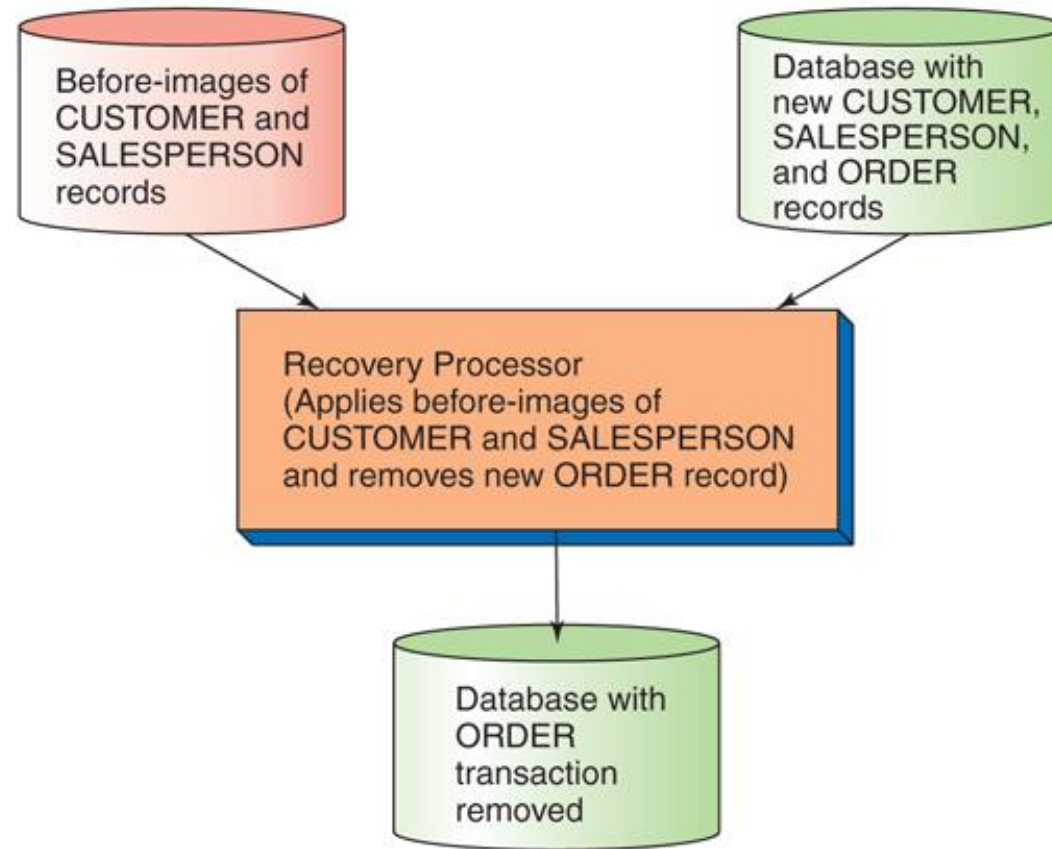
Accept order data from browser.
Read CUSTOMER and SALESPERSON records.
Change CUSTOMER and SALESPERSON records.
Rewrite CUSTOMER record.
Rewrite SALESPERSON record. } (Log records written here)
Insert new ORDER record.

****CRASH****

(a) Processing with a Problem

Figure 6-24: Recovery Example

Recovery Example I



(b) Recovery Processing

Figure 6-24: Recovery Example (Cont'd)



Backing Up the HSD Database Microsoft SQL Server 2014

The **Back Up Database – HSD** dialog box

The **HSD** database

The **SIMPLE** recovery model

The **Full** backup type

The database itself is being backed up

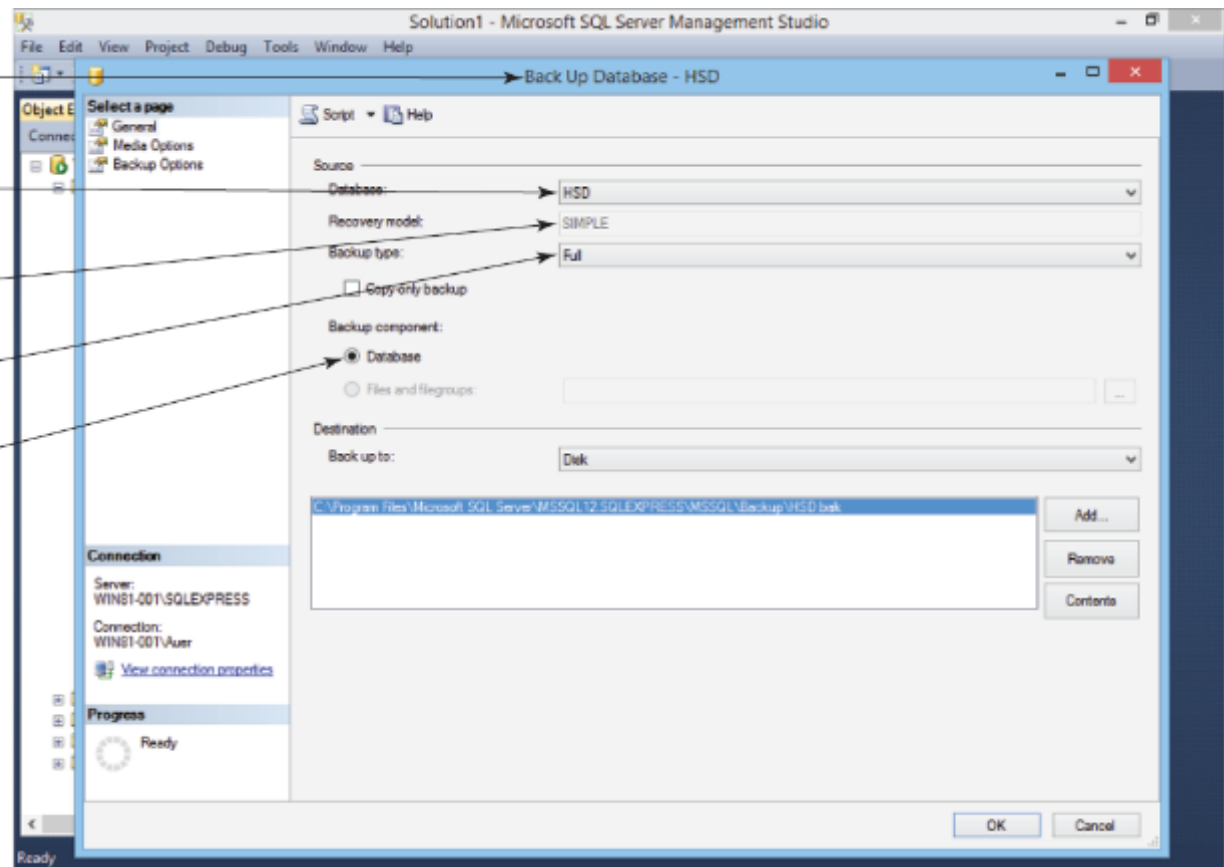


Figure 6-25: Backing Up the HSD Database



Additional DBA Responsibilities

- The DBA needs to ensure that a system exists to gather and record user reported errors and other problems.
 - A means needs to be devised to prioritize those errors and problems and to ensure that they are corrected accordingly .
- The DBA needs to create and manage a process for controlling the database configuration.
 - Procedures for recording change requests
 - Conducting user and developer reviews of such requests
 - Creating projects and tasks
- The DBA is responsible for ensuring that appropriate documentation is maintained.
 - Database structure
 - Concurrency control
 - Security
 - Backup and recovery
 - Applications used



END OF CHAPTER 7

